ConfigStream

Scalable Configuration Change Storage and Streaming Analytics

Phạm Công Hoàng Lê Tiến Hợp
Mai Vũ Hoàng Nguyễn Đức Dũng
Group 4 · Big Data Storage & Processing

First milestone checkpoint · 26 October 2026



Khoảng 25 giây. Chào thầy và các bạn. Nhóm 4 trình bày ConfigStream, hệ thống lưu trữ và phân tích thay đổi cấu hình. Buổi checkpoint này tập trung vào bài toán, dữ liệu, kiến trúc và những phần nhóm đã triển khai được. Chưa trình bày như một sản phẩm đã hoàn thiện hoặc đã chứng minh khả năng mở rộng.

The problem: configuration changes over time

Router configuration: version 10

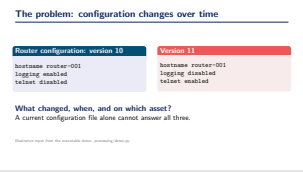
```
hostname router-001
logging enabled
telnet disabled
```

Version 11

```
hostname router-001
logging disabled
telnet enabled
```

What changed, when, and on which asset?
A current configuration file alone cannot answer all three.

Illustrative input from the executable demo: processing/demo.py.



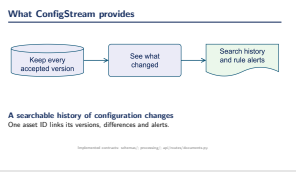
Khoảng 45 giây. Mở đầu bằng ví dụ cụ thể: cùng một router nhưng phiên bản sau tắt logging và bật telnet. Đây là dữ liệu minh hoạ do nhóm tạo, không phải sự cố thực tế. Nếu chỉ giữ file hiện tại, ta không biết thay đổi xảy ra lúc nào và khác phiên bản trước ra sao. Nhóm muốn lưu lịch sử và chỉ ra phần thay đổi thay vì chỉ giữ trạng thái cuối cùng.

What ConfigStream provides



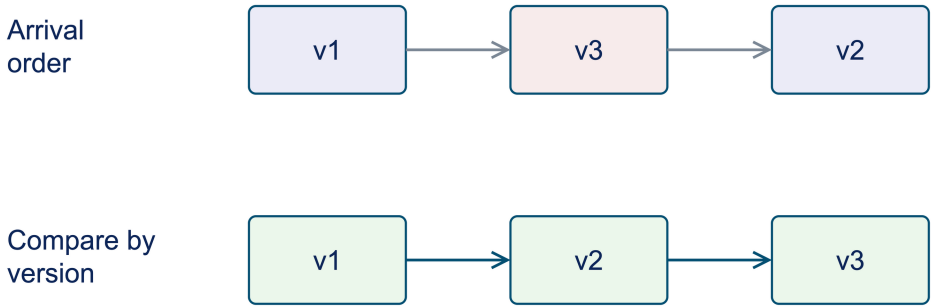
A searchable history of configuration changes
One asset ID links its versions, differences and alerts.

Implemented contracts: schemas/; processing/; api/routes/documents.py.



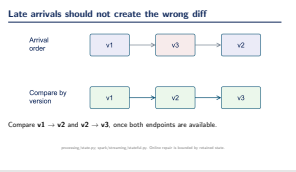
Khoảng 35 giây. Giải thích ba việc hệ thống làm: lưu phiên bản đã được chấp nhận, so sánh các phiên bản và tra cứu lịch sử cùng cảnh báo theo rule. Asset là đối tượng như router hoặc một file cấu hình. ID của asset giúp nối lịch sử lại với nhau. Cảnh báo là kết quả rule, không đồng nghĩa với một sự cố bảo mật đã được xác nhận.

Late arrivals should not create the wrong diff



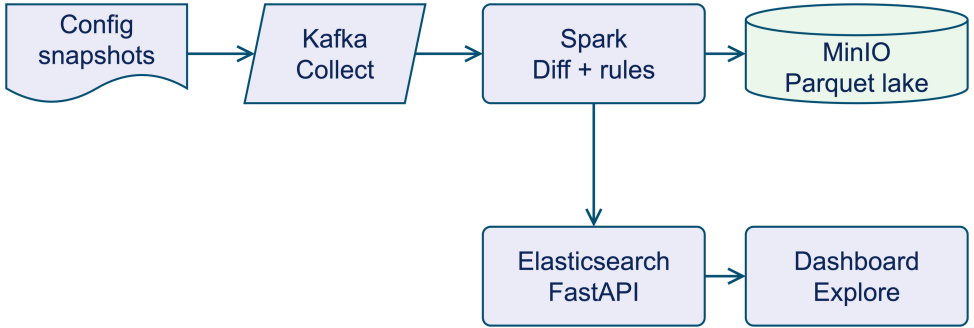
Compare **v1** → **v2** and **v2** → **v3**, once both endpoints are available.

processing/state.py; spark/streaming/stateful.py. Online repair is bounded by retained state.



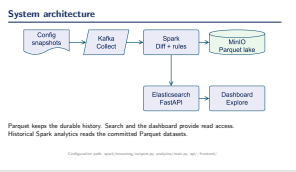
Khoảng 55 giây. Chỉ vào hàng trên: dữ liệu có thể đến theo thứ tự 1, 3, 2. Nếu so sánh theo thứ tự đến, ta sẽ tạo cặp 1 sang 3 và bỏ qua phiên bản 2. Hàng dưới là cách nhóm xử lý: giữ các phiên bản theo asset và chỉ tạo diff giữa các phiên bản liền kề. Nếu 2 đến khi state còn giữ được 1 và 3 thì tạo đủ hai cặp đúng. State có giới hạn lưu giữ, nên không hứa sửa được mọi dữ liệu đến muộn vô hạn.

System architecture



Parquet keeps the durable history. Search and the dashboard provide read access.
Historical Spark analytics reads the committed Parquet datasets.

Configuration path: spark/streaming/outputs.py; analytics/main.py; api/; frontend/.



Khoảng 65 giây. Đi theo mũi tên: snapshot đi qua Kafka, Spark chuẩn hoá và so sánh phiên bản, rồi ghi lịch sử vào MinIO dưới dạng Parquet. Nhánh bên dưới ghi metadata vào Elasticsearch, FastAPI cung cấp API và dashboard hiển thị. Lịch sử Parquet là dữ liệu gốc để đọc lại; search là lớp phục vụ truy vấn. Trong code, lake được ghi trước search. Sơ đồ rút gọn đường cấu hình; các bộ dữ liệu RPKI, RIS, DNS có pipeline riêng và không bị ép qua bộ so sánh cấu hình này. Nếu thầy hỏi analytics batch, nó đọc Parquet đã commit.

Data is ready for development and replay

Controlled examples

A deterministic generator creates snapshots, retries and late arrivals. Useful for checking whether the result is correct.

TerraDS configuration history

6,505	file histories
10,188	accepted versions
3,683	adjacent version pairs

Public Git history of infrastructure code.

Large datasets stay outside Git; source manifests and result receipts are versioned.

generator/; workloads/results/terrads/history-summary.json. Git changes are not deployment events.

Khoảng 50 giây. Nhóm có hai loại dữ liệu. Bộ tạo dữ liệu giúp chủ động kiểm tra thứ tự đảo, dữ liệu lặp và các thay đổi biết trước. TerraDS là lịch sử mã hạ tầng từ Git: 6.505 lịch sử file, 10.188 phiên bản và 3.683 cặp liền kề được chấp nhận. Không gọi đây là 10.188 lần triển khai ngoài thực tế. Dữ liệu lớn không đưa hết vào Git; repo giữ nguồn gốc, manifest và kết quả. Nếu được hỏi thêm, nhóm còn có dữ liệu RPKI, RIS và DNS, nhưng hôm nay tập trung đường xử lý cấu hình để dễ theo dõi.

Implemented: differences and rule alerts

Diff: version 10 → 11

```
- logging enabled
- telnet disabled
+ logging disabled
+ telnet enabled
```

Rules detect

Logging disabled
High severity

Telnet enabled
Critical severity

Repeated delivery must not invent another logical change.

processing/demo.py; processing/diff.py; rules/default.yaml; tests/unit/test_rules.py.

Khoảng 45 giây. Quay lại ví dụ đầu: output chỉ ra các dòng bị bỏ và thêm. Rule mặc định báo logging bị tắt ở mức high và telnet được bật ở mức critical. Đây là rule minh hoạ đã có trong code, không phải mô hình AI dự đoán. Chuẩn hoá xử lý các khác biệt văn bản được hỗ trợ; không phải parser hiểu mọi ý nghĩa cấu hình. Việc gửi lại cùng snapshot cũng không được tạo thêm thay đổi logic mới.

Executable code is available in Git

Repository

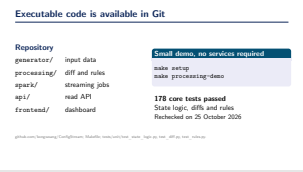
`generator/`	input data
`processing/`	diff and rules
`spark/`	streaming jobs
`api/`	read API
`frontend/`	dashboard

Small demo, no services required

```
make setup
make processing-demo
```

178 core tests passed
State logic, diffs and rules
Rechecked on 25 October 2026

github.com/kongwoang/ConfigStream; Makefile; tests/unit/test_state_logic.py, test_diff.py, test_rules.py.



Khoảng 50 giây. Mở link repo nếu có mạng hoặc chuyển terminal đã chuẩn bị. Hai lệnh trên chạy demo xử lý thuần Python, không cần dựng Kafka hay MinIO. Không dùng demo nhỏ này để nói cả stack đang online. Nhóm vừa kiểm tra 178 test của state logic, diff và rule. CI của revision mã nguồn 3980ce1 đã đạt trước khi thêm slide. Nếu thầy muốn xem thực thi, chạy make processing-demo; phần output chính chính là ví dụ vừa giải thích.

A complete local replay has been recorded



29.4 MB of raw and processed Parquet committed.

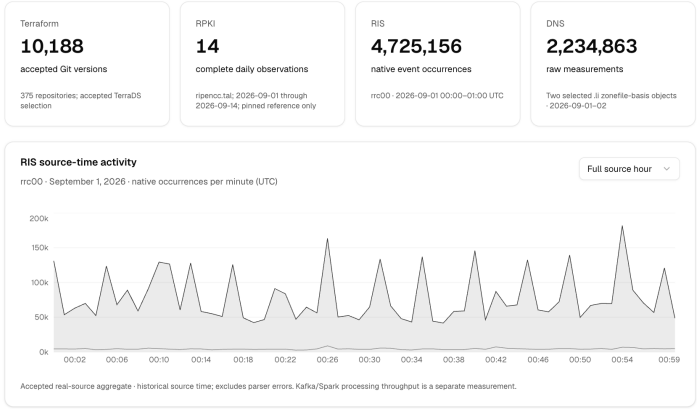
This demonstrates the integrated path on the selected local workload.

Recorded acceptance: docs/phase12-acceptance.md. 29,402,997 bytes; not a new replay or a scale benchmark.

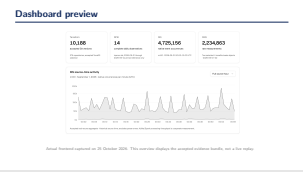
A complete local replay has been recorded
10,188 snapshots
Kafka + Spark Local replay
3,683 adjacent diffs
29.4 MB of raw and processed Parquet committed.
This demonstrates the integrated path on the selected local workload.

Khoảng 40 giây. Đây là kết quả replay đã được ghi nhận trong repo, không phải lần chạy mới trong buổi thuyết trình. 10.188 snapshot qua đường tích hợp và cho 3.683 diff liền kề. Khoảng 29,4 MB là tổng Parquet raw và processed đã commit. Chỉ kết luận đường tích hợp chạy được trên phạm vi local đã thử; không kết luận Spark nhanh hơn baseline hay hệ thống đã đạt quy mô production. Cấu hình replay lịch sử có retention riêng, khác mặc định demo online.

Dashboard preview



Actual frontend captured on 25 October 2026. This overview displays the accepted evidence bundle, not a live replay.



Khoảng 40 giây. Đây là ảnh giao diện thật được mở từ frontend hiện tại. Trang overview tóm tắt dữ liệu đã được chấp nhận ở các miền. Phần lịch sử cấu hình có API để tra cứu theo asset và thời gian khi backend chạy. Các con số trên ảnh thuộc những đơn vị khác nhau, không cộng lại thành một chỉ số tổng. Ảnh này không chứng minh backend đang chạy trực tiếp trong buổi checkpoint.

Checkpoint readiness

Requirement	What the group can show
Git + executable code	Repository, runnable demo and automated tests
Data ready	Controlled inputs, TerraDS history and source manifests
Architecture ready	Kafka, Spark, Parquet, search API and dashboard

The core workflow is implemented and demonstrated locally

Checkpoint readiness

Requirement	What the group can show
Git + executable code	Repository, runnable demo and automated tests
Data ready	Controlled inputs, TerraDS history and source manifests
Architecture ready	Kafka, Spark, Parquet, search API and dashboard

The core workflow is implemented and demonstrated locally

Khoảng 40 giây. Trả lời trực tiếp ba yêu cầu thầy đưa ra. Git đã có code chạy được; dữ liệu và manifest đã chuẩn bị; kiến trúc đã có implementation và minh chứng local. Chỉ vào đúng slide trước tương ứng mỗi hàng. Chưa gọi đây là bài hoàn thành cuối kỳ; vẫn cần củng cố demo tích hợp, đo đạc và cách trình bày kết quả.

Next milestone

1. **Rehearse the integrated demo**
   Start the services, send a change, inspect the stored result.
2. **Evaluate larger workloads**
   Measure throughput, latency and storage with clear units.
3. **Strengthen recovery evidence**
   Repeat failure and restart checks for the latest revision.

Current evidence is local / single-node; distributed scale and HA remain open.

Thank you — questions?

Next milestone
1. Rehearse the integrated demo
Start the services, send a change, inspect the stored result.
2. Evaluate larger workloads
Measure throughput, latency and storage with clear units.
3. Strengthen recovery evidence
Repeat failure and restart checks for the latest revision.
Current evidence is local / single-node; distributed scale and HA remain open.
Thank you — questions?

Khoảng 40 giây. Ba việc nhóm dự kiến ưu tiên tiếp: diễn tập demo tích hợp từ đầu, mở rộng đánh giá với đơn vị đo rõ ràng và chạy lại kiểm thử phục hồi trên revision mới nhất. Không nói những việc dự kiến này đã hoàn thành. Bằng chứng hiện có chủ yếu local hoặc single-node; chưa chứng minh HA hay horizontal scalability. Kết thúc và mời thầy góp ý về phạm vi dữ liệu và ưu tiên cho milestone tiếp theo.